

Théorie des Graphes

TP PRATIQUE 1

RÉSEAUVAK

Représentations de Graphes, BFS & DFS

VOTRE MISSION

Vous êtes développeur junior chez **ESGC Connect**, la startup qui développe le réseau social interne de l'ESGC-VAK. L'application permet aux étudiants de se connecter entre filières (GI, GC, TOPO). Votre mission : construire le **moteur d'analyse du réseau social** — découvrir qui connaît qui, trouver le degré de séparation entre deux étudiants, détecter les groupes isolés, et analyser les dépendances entre cours.

Approche pédagogique : on construit d'abord la structure du graphe, puis on implémente BFS pour explorer en largeur ("qui est le plus proche ?"), puis DFS pour explorer en profondeur ("est-ce qu'il y a un cycle ?"). Chaque étape réutilise les précédentes.

Durée : 3 heures en salle machine | **Langage :** C++ (compilateur g++ avec -std=c++17)

ARBORESCENCE DU PROJET

TP1_ReseauVAK/	
include/	Fichiers d'en-tête
graph.h	Classe Graph (à compléter)
bfs.h	Déclarations BFS
dfs.h	Déclarations DFS
src/	Code source
graph.cpp	Implémentation Graph (à compléter)
bfs.cpp	Implémentation BFS (à compléter)
dfs.cpp	Implémentation DFS (à compléter)
main.cpp	Programme principal (fourni)
data/	Graphes de test
mini_test.txt	Petit graphe debug (6 sommets)
reseau_vak.txt	Réseau social (non orienté, 12 sommets)
mega_campus.txt	Stress test (50 sommets)
prerequis.txt	Prérequis de cours (orienté, DAG)
prerequis_cycle.txt	Prérequis avec cycle
tests/	
test_tp1.cpp	20 tests automatisés
docs/	
ENONCE.md	Résumé de l'énoncé
BAREME.md	Grille de notation
Makefile	Compilation automatique
README.md	Documentation du projet

LES 6 ÉTAPES — PROGRESSION

Étape	Objectif	Points	Philosophie
1	Classe Graph : liste d'adjacence	3	La fondation : représenter un graphe
2	Matrice d'adjacence & affichage	2	Voir les deux représentations côte à côte
3	BFS : parcours en largeur	5	Explorer couche par couche
4	Composantes connexes (via BFS)	3	Trouver les groupes isolés
5	DFS : parcours en profondeur	5	Plonger le plus loin possible
6	Tri topologique & détection de cycles	4	Ordonner les cours, détecter les boucles

CODE	Qualité code + commentaires	+3	Noms, explications, indentation
TOTAL		25	Progression garantie !

POURQUOI 6 ÉTAPES ET PAS 3 ?

On pourrait coder BFS et DFS directement d'un bloc. Mais vous ne comprendriez pas **pourquoi** chaque brique existe. En procédant étape par étape, vous allez :

- ① D'abord construire la fondation (étape 1) — sans graphe, aucun algorithme ne tourne
- ② Puis voir votre graphe sous deux formes (étape 2) — moment "aha, c'est les mêmes données !"
- ③ Puis explorer en largeur (étape 3) — le cœur : une file, des distances, des couches
- ④ Puis exploiter BFS (étape 4) — chemins, composantes, degrés de séparation
- ⑤ Puis explorer en profondeur (étape 5) — même graphe, autre stratégie, résultats différents
- ⑥ Enfin appliquer DFS à un vrai problème (étape 6) — tri topologique, détection de cycles

Chaque paire d'étapes suit le même rythme : d'abord le mécanisme, puis les applications. Étapes 1-2 = Graph (construire, voir). Étapes 3-4 = BFS (parcourir, appliquer). Étapes 5-6 = DFS (parcourir, appliquer).

DEADLINE : Fin de la séance (3h) | **Squelette fourni** : TP1_ReseauVAK_code.zip (headers, TODOs, Makefile, tests, 5 graphes de test)

COMPRENDRE AVANT DE CODER

LE RÉSEAU SOCIAL COMME GRAPHE

Un réseau social est naturellement un graphe : chaque **étudiant** est un sommet, chaque **amitié** est une arête. Si Dossou et Agbangla sont amis, on a l'arête (Dossou, Agbangla). L'amitié est réciproque, donc le graphe est **non orienté**.

L'ANALOGIE WHATSAPP

Imaginez que Dossou poste un message dans le groupe WhatsApp de sa filière. Ses amis directs le voient immédiatement (**couche 1**). Puis les amis de ses amis le voient via un transfert (**couche 2**). Et ainsi de suite, le message se propage en ondes concentriques. C'est **exactement** ce que fait BFS.

```
Le message de Dossou se propage :
Couche 0 : Dossou lui-même
Couche 1 : Agbangla, Hounenou, Adjovi      (amis directs)
Couche 2 : Amoussou, Mensah, Kodjo        (amis d'amis)
Couche 3 : Zannou                          (via Amoussou ET Kodjo)
Couche 4 : Kiki, Hounkpatin                (via Zannou)
```

Sossa et Viho ne reçoivent JAMAIS le message → composantes séparées !

DFS, lui, c'est comme si Dossou envoyait le message à UN seul ami, qui le transmet à UN seul de ses amis, etc. — on plonge le plus profondément possible avant de revenir en arrière. Résultat différent, même graphe.

Format des fichiers de données (exemple : reseau_vak.txt) :

```
12 14 // ligne 1 : V sommets, E arêtes
Dossou Agbangla Hounenou ... // ligne 2 : noms des sommets
0 1 // Dossou -- Agbangla (une arête par ligne)
0 2 // Dossou -- Hounenou
... // (voir data/reseau_vak.txt pour la suite)
```

Le graphe contient : 3 clusters de filières (GI : 4 étudiants, GC : 3, TOPO : 3) reliés par 3 ponts inter-filières, + 2 étudiants isolés (Sossa et Viho). Total : 12 sommets, 14 arêtes, 3 composantes connexes.

Exercice 0 (avant de coder) : ouvrez data/reseau_vak.txt et DESSINEZ le graphe à la main sur papier. Identifiez les 3 clusters, les ponts, et les isolés. C'est votre meilleur outil de debug pour tout le TP !

DEUX FAÇONS DE REPRÉSENTER

Un même graphe peut se stocker de deux façons différentes, chacune avec ses avantages :

	Liste d'adjacence	Matrice d'adjacence
Structure	vector<vector<int>>	vector<vector<int>>
Mémoire	$O(V + E)$ — économe	$O(V^2)$ — coûteux si creux
Voisins de u ?	$O(\text{degré}(u))$ — direct	$O(V)$ — parcourir la ligne
Arête (u,v) existe ?	$O(\text{degré}(u))$ — chercher	$O(1)$ — <code>adj[u][v]</code>
Idéal pour	Graphes creux (peu d'arêtes)	Graphes denses / test rapide

Pour ESGC Connect : 12 étudiants × 14 amitiés = graphe creux. La liste d'adjacence est le bon choix. Mais on implémentera les deux pour comparer !

STRUCTURES C++ (référence rapide — voir include/*.h)

```
// Classe Graph : voir include/graph.h
// Attributs : V (sommets), directed, adjList (liste d'adjacence), names
// Méthodes : addEdge, loadFromFile, neighbors, getAdjMatrix, print...

struct BFSResult { vector<int> distance, parent, order; };
struct DFSResult { vector<int> discovery, finish, parent, order; };
```

ÉTAPE 1 : CLASSE GRAPH — LISTE D'ADJACENCE [3 points]

Objectif : construire la fondation. Sans un graphe bien représenté, aucun algorithme ne fonctionne.

FONCTION 1.1 : Constructeur Graph(int V, bool directed)

Objectif : initialiser un graphe vide à V sommets.

```
Graph::Graph(int V, bool directed) : V(V), directed(directed), adjList(V), names(V) {
    // TODO 1 : Initialiser names[i] = to_string(i) pour tout i
    //          (noms par défaut : "0", "1", "2", ...)
}
```

FONCTION 1.2 : addEdge(int u, int v)

Objectif : ajouter une arête entre u et v. Si le graphe est non orienté, ajouter dans les deux sens.

```
void Graph::addEdge(int u, int v) {
    // TODO 1 : Ajouter v dans adjList[u]
    // TODO 2 : Si le graphe est NON orienté (!directed),
    //          ajouter aussi u dans adjList[v]
}
```

Attention : pour un graphe non orienté, chaque arête apparaît DEUX FOIS dans la liste d'adjacence. L'arête Dossou—Agbangla donne : adjList[0] contient 1, ET adjList[1] contient 0.

FONCTION 1.3 : loadFromFile(const string& filename)

Objectif : charger un graphe depuis un fichier texte. Format :

```
Ligne 1 : V E          (nombre de sommets, nombre d'arêtes)
Ligne 2 : nom0 nom1 ... nomV-1 (noms séparés par des espaces)
Lignes suivantes : u v      (une arête par ligne)

void Graph::loadFromFile(const string& filename) {
    // TODO 1 : Ouvrir le fichier avec ifstream
    //          Si échec, afficher une erreur et retourner

    // TODO 2 : Lire V et E

    // TODO 3 : Redimensionner adjList et names
    //          adjList.assign(V, {});
    //          names.resize(V);

    // TODO 4 : Lire les V noms

    // TODO 5 : Lire les E arêtes et appeler addEdge(u, v) pour chacune
}
```

FONCTION 1.4 : neighbors(int u) et accesseurs

```
const vector<int>& Graph::neighbors(int u) const {
    // TODO : Retourner adjList[u]
}

int Graph::getV() const { /* TODO : retourner V */ }
bool Graph::isDirected() const { /* TODO : retourner directed */ }
const string& Graph::getName(int u) const { /* TODO : retourner names[u] */ }
```

À ce stade : votre graphe se charge et stocke les connexions. Testez avec **make test**. Les tests de l'étape 1 doivent passer.

ÉTAPE 2 : MATRICE D'ADJACENCE & AFFICHAGE [2 points]

Objectif : voir les deux représentations côte à côte. La liste est compacte, la matrice est lisible. Les deux encodent la même information.

FONCTION 2.1 : getAdjMatrix()

Objectif : construire la matrice d'adjacence à partir de la liste d'adjacence.

```
vector<vector<int>> Graph::getAdjMatrix() const {
    // TODO 1 : Créer une matrice VxV initialisée à 0
    //           vector<vector<int>> matrix(V, vector<int>(V, 0));
    // TODO 2 : Pour chaque sommet u de 0 à V-1 :
    //           Pour chaque voisin v dans adjList[u] :
    //               matrix[u][v] = 1
    // TODO 3 : Retourner matrix
}
```

FONCTION 2.2 : printAdjList()

```
void Graph::printAdjList() const {
    // TODO : Pour chaque sommet u :
    //   Afficher : "Dossou (0) : Agbangla Hounénou Adjovi"
    //   Format : nom (indice) : voisin1 voisin2 ...
}
```

FONCTION 2.3 : printAdjMatrix()

```
void Graph::printAdjMatrix() const {
    // TODO 1 : Récupérer la matrice via getAdjMatrix()

    // TODO 2 : Afficher la ligne d'en-tête
    //           (utiliser les 3 premiers caractères : names[i].substr(0, 3))

    // TODO 3 : Pour chaque ligne, afficher le nom (3 car.) puis les valeurs
    //           Utiliser setw(4) de <iomanip> pour aligner
}
```

Exemple de sortie pour printAdjList() :

```
Dossou (0) : Agbangla Hounenou Adjovi
Agbangla (1) : Dossou Hounenou Amoussou
... // (12 lignes au total)
Sossa (10) : // ← liste VIDE (isolé)
```

Checkpoint étape 2 : make test — les tests 6 à 8 doivent passer. Vérifiez que les isolés ont une liste vide et que la matrice est cohérente avec la liste.

5 graphes de test sont fournis dans data/ (de 6 à 50 sommets). Voir le README dans le ZIP pour les détails de chaque fichier.

ÉTAPE 3 : BFS — PARCOURS EN LARGEUR [5 points]

Objectif : explorer le graphe couche par couche, comme une onde qui se propage. BFS trouve le plus court chemin en nombre d'arêtes.

L'IDÉE : L'ONDE QUI SE PROPAGE

On va d'abord dérouler BFS sur le petit graphe `mini_test.txt` (6 sommets). C'est plus facile à suivre que `reseau_vak.txt`.

```
Rappel mini_test.txt :
Afi(0) — Beno(1) — Dina(3) — Elie(4)
|      /      |      /
Cedric(2) — Fara(5) ● isolé
```

BFS depuis Afi (0) :

```
Couche 0 : Afi                → distance = 0
Couche 1 : Beno, Cedric       → distance = 1
Couche 2 : Dina               → distance = 2
Couche 3 : Elie               → distance = 3

Fara n'est JAMAIS atteint → distance = -1 (inaccessible)
```

LA FILE (QUEUE) : LE SECRET DE BFS

BFS utilise une **file FIFO** (First In, First Out) : on traite les sommets dans l'ordre où on les découvre. C'est ce qui garantit l'exploration couche par couche.

```
#include <queue>
queue<int> q;
q.push(x);          // ajouter à la fin
int u = q.front(); q.pop(); // retirer du début
```

FONCTION 3.1 : `bfs(const Graph& g, int source)`

```
BFSResult bfs(const Graph& g, int source) {
    int V = g.getV();
    BFSResult result;
    result.distance.assign(V, -1);    // -1 = non visité
    result.parent.assign(V, -1);     // -1 = pas de parent

    // TODO 1 : Initialiser la source
    // result.distance[source] = 0
    // Créer une queue<int> et y ajouter source
    // Ajouter source à result.order

    // TODO 2 : Boucle BFS
    // Tant que la queue n'est pas vide :
    //   u = front + pop
    //   Pour chaque voisin v de u (g.neighbors(u)) :
    //     Si distance[v] == -1 (non visité) :
    //       distance[v] = distance[u] + 1
    //       parent[v] = u
    //       Ajouter v à la queue
    //       Ajouter v à result.order

    return result;
}
```

}

FONCTION 3.2 : getPath(const BFSResult& result, int target, const Graph& g)

Objectif : reconstruire le chemin de la source vers target en remontant les parents.

```
string getPath(const BFSResult& result, int target, const Graph& g) {
    // TODO 1 : Si result.distance[target] == -1, retourner "Aucun chemin"

    // TODO 2 : Remonter parent[] depuis target jusqu'à la source
    //             vector<int> path;
    //             int current = target;
    //             Tant que current != -1 :
    //                 path.push_back(current)
    //                 current = result.parent[current]

    // TODO 3 : Inverser le chemin (il est à l'envers !)

    // TODO 4 : Construire la string "Dossou -> Adjovi -> Mensah"
    //             en utilisant g.getName()
}
```

DÉROULEMENT SUR MINI_TEST.TXT

BFS depuis Afi (source = 0), étape par étape :

Étape	Queue	u traité	Voisins non visités	distance[]
Init	[0]	—	—	[0, -1, -1, -1, -1, -1]
1	[1, 2]	0 (Afi)	1, 2	[0, 1, 1, -1, -1, -1]
2	[2, 3]	1 (Beno)	3	[0, 1, 1, 2, -1, -1]
3	[3]	2 (Cedric)	(tous visités)	inchangé
4	[4]	3 (Dina)	4	[0, 1, 1, 2, 3, -1]
5	[]	4 (Elie)	(aucun)	[0, 1, 1, 2, 3, -1]

Résultat : distance[5] (Fara) reste à -1. BFS ne peut pas l'atteindre depuis Afi. Ce sont deux composantes connexes distinctes ! Sur reseau_vak.txt, il y aura 3 composantes à trouver.

Note sur l'ordre de visite : l'ordre dans lequel BFS explore les voisins dépend de l'ordre dans adjList (= l'ordre d'insertion). Les tests vérifient les DISTANCES et les COMPOSANTES, pas l'ordre exact de visite.

ÉTAPE 4 : COMPOSANTES CONNEXES [3 points]

Objectif : trouver TOUS les groupes d'étudiants qui ne sont pas connectés entre eux. On relance BFS depuis chaque sommet non visité.

L'IDÉE : RELANCER BFS

```
Composante 1 : {Dossou, Agbangla, Hounenou, Amoussou, Adjovi,
                Mensah, Kodjo, Zannou, Kiki, Hounkpatin} → 10 étudiants
Composante 2 : {Sossa} → 1 étudiant (isolé)
Composante 3 : {Viho} → 1 étudiant (isolé)

→ 3 composantes connexes dans le réseau ESGC-VAK
```

FONCTION 4.1 : `connectedComponents(const Graph& g)`

```
vector<vector<int>> connectedComponents(const Graph& g) {
    int V = g.getV();
    vector<bool> visited(V, false);
    vector<vector<int>> components;

    // TODO : Pour chaque sommet u de 0 à V-1 :
    //   Si !visited[u] :
    //       Lancer BFS depuis u
    //       Marquer tous les sommets de result.order comme visited
    //       Ajouter result.order comme nouvelle composante

    return components;
}
```

FONCTION 4.2 : `printComponents(...)`

```
void printComponents(const vector<vector<int>>& comps, const Graph& g) {
    // TODO : Pour chaque composante, afficher :
    //   "Composante 1 (10 étudiants) : Dossou, Agbangla, Hounenou, ..."
    //   "Composante 2 (1 étudiant) : Sossa"
    //   "Composante 3 (1 étudiant) : Viho"
}
```

FONCTION 4.3 : `degreeSeparation(const Graph& g, int u, int v)`

Objectif : trouver le degré de séparation entre deux étudiants (nombre minimum d'amitiés à traverser).

```
int degreeSeparation(const Graph& g, int u, int v) {
    // TODO : Lancer BFS depuis u, retourner result.distance[v]
    //          (-1 si pas dans la même composante)
}
```

Exemples attendus :

```
degreeSeparation(g, 0, 8) = 4 // Dossou → ... → Zannou → Kiki : 4 amitiés
degreeSeparation(g, 0, 10) = -1 // Dossou → Sossa : impossible
degreeSeparation(g, 4, 9) = 3 // Adjovi(4) → Kodjo(6) → Zannou(7) → Hounkpatin(9)
```

Checkpoint étape 4 : make test — les tests 13 à 16 doivent passer. Votre programme trouve 3 composantes et les distances sont correctes.

ÉTAPE 5 : DFS — PARCOURS EN PROFONDEUR [5 points]

Objectif : explorer en plongeant le plus profondément possible avant de revenir en arrière (backtracking). DFS utilise une pile (ou la récursion) au lieu d'une file.

LA DIFFÉRENCE BFS vs DFS

Sur mini_test.txt depuis Afi (0) :

BFS (file FIFO) : explore en LARGEUR → Afi, Beno, Cedric, Dina, Elie
 DFS (pile/récursion) : explore en PROFONDEUR → Afi, Beno, Cedric, Dina, Elie
 (même résultat ici, mais ordre différent sur des graphes plus grands !)

BFS = "onde" (couche par couche) → trouve les DISTANCES

DFS = "plongée" (descendre, remonter) → trouve les TEMPS (discovery/finish)

TEMPS DE DÉCOUVERTE ET DE FIN

DFS maintient un **compteur de temps** global. Pour chaque sommet v : $discovery[v]$ = moment où on le découvre, $finish[v]$ = moment où on a fini d'explorer tous ses voisins.

DFS depuis Afi (0) sur mini_test.txt :

Temps 0 : Découvre Afi	→ $discovery[0] = 0$
Temps 1 : Découvre Beno	→ $discovery[1] = 1$
Temps 2 : Découvre Cedric	→ $discovery[2] = 2$
Temps 3 : Découvre Dina	→ $discovery[3] = 3$
Temps 4 : Découvre Elie	→ $discovery[4] = 4$
Temps 5 : Finit Elie	→ $finish[4] = 5$
Temps 6 : Finit Dina	→ $finish[3] = 6$
Temps 7 : Finit Cedric	→ $finish[2] = 7$
Temps 8 : Finit Beno	→ $finish[1] = 8$
Temps 9 : Finit Afi	→ $finish[0] = 9$

Fara (5) : $discovery = -1$, $finish = -1$ (non visité)

FONCTION 5.1 : dfsVisit

Implémentez DFS avec une approche récursive. La fonction auxiliaire `dfsVisit` explore depuis un sommet.

```
void dfsVisit(const Graph& g, int u, vector<bool>& visited,
             DFSResult& result, int& timer) {
    // TODO 1 : Marquer u comme visité : visited[u] = true
    // TODO 2 : Enregistrer le temps de découverte : result.discovery[u] = timer++
    // TODO 3 : Ajouter u à result.order

    // TODO 4 : Pour chaque voisin v de u (g.neighbors(u)) :
    //             Si !visited[v] :
    //                 result.parent[v] = u
    //                 Appel récursif : dfsVisit(g, v, visited, result, timer)

    // TODO 5 : Enregistrer le temps de fin : result.finish[u] = timer++
}
```

FONCTION 5.2 : dfs(const Graph& g, int source)

Objectif : lancer DFS depuis source, ne visite que les sommets accessibles.

```
DFSResult dfs(const Graph& g, int source) {
    int V = g.getV();
    DFSResult result;
    result.discovery.assign(V, -1);
    result.finish.assign(V, -1);
    result.parent.assign(V, -1);
```

```
vector<bool> visited(V, false);
int timer = 0;

// TODO : Appeler dfsVisit(g, source, visited, result, timer)
return result;
}
```

FONCTION 5.3 : dfsComplete(const Graph& g)

Objectif : relancer DFS depuis chaque sommet non visité. Utile pour le tri topologique (un graphe orienté peut avoir plusieurs sources).

```
DFSResult dfsComplete(const Graph& g) {
    // (même initialisation que dfs : result, visited, timer)

    // TODO : Pour chaque sommet u de 0 à V-1 :
    //         Si !visited[u] :
    //             dfsVisit(g, u, visited, result, timer)

    return result;
}
```

Piège classique : dans un graphe NON orienté, DFS verra le parent de u comme voisin. Il faut vérifier !visited[v], pas juste v != parent[u]. Les deux approches marchent ici, mais visited[] est plus sûr.

Checkpoint étape 5 : make test — les tests 17 et 18 doivent passer. DFS visite les bons sommets et discovery[v] < finish[v] pour tout sommet visité.

ÉTAPE 6 : TRI TOPOLOGIQUE & DÉTECTION DE CYCLES [4 points]

Objectif : on passe à un graphe ORIENTÉ. Les prérequis entre cours forment un DAG (si tout va bien). DFS permet de les ordonner ET de détecter des dépendances circulaires.

CHANGEMENT DE DÉCOR : DU RÉSEAU SOCIAL AUX COURS

Votre manager chez ESGC Connect a une nouvelle demande : "On veut aussi aider les étudiants à planifier l'ordre de leurs cours. Certains cours ont des prérequis. Peut-on trouver un ordre valide ?" C'est un graphe **orienté** : Maths → Algo signifie qu'il faut avoir fait Maths AVANT Algo. L'arête a un sens !

NOUVEAU SCÉNARIO : LES PRÉREQUIS DE COURS

Le fichier prerequis.txt (voir data/) contient 8 cours et 9 prérequis. Exemples : Maths → Algo, Info1 → BDD, Graphes → Projet. **Question :** dans quel ordre suivre les cours pour respecter tous les prérequis ? C'est le **tri topologique**.

FONCTION 6.1 : topologicalSort(const Graph& g)

Objectif : retourner les sommets dans un ordre tel que pour chaque arête $u \rightarrow v$, u apparaît avant v .

```
vector<int> topologicalSort(const Graph& g) {
    // TODO 1 : Lancer dfsComplete(g) pour obtenir finish[] de tous les sommets

    // TODO 2 : Créer un vecteur d'indices {0, 1, 2, ..., V-1}

    // TODO 3 : Trier ce vecteur par finish[] DÉCROISSANT :
    //             sort(indices.begin(), indices.end(),
    //                 [&result](int a, int b) {
    //                     return result.finish[a] > result.finish[b];
    //                 });

    // TODO 4 : Retourner le vecteur trié
}
```

Résultat attendu pour prerequis.txt :

```
Ordre topologique : Maths → Info1 → Algo → Réseaux → BDD → Graphes → Systèmes →
Projet
(ou toute permutation valide respectant les prérequis)
```

FONCTION 6.2 : hasCycle(const Graph& g)

Objectif : détecter si un graphe orienté contient un cycle (dépendance circulaire).

```
bool hasCycle(const Graph& g) {
    int V = g.getV();
    // États : 0 = blanc (non visité), 1 = gris (en cours), 2 = noir (terminé)
    vector<int> color(V, 0);

    // TODO : Pour chaque sommet u non visité (color == 0) :
    // Appeler dfsCycleVisit(g, u, color)
    // Si elle retourne true → cycle détecté !
}

bool dfsCycleVisit(const Graph& g, int u, vector<int>& color) {
    // TODO 1 : color[u] = 1 (gris – en cours d'exploration)
    // TODO 2 : Pour chaque voisin v de u :
    // Si color[v] == 1 → CYCLE ! (on retombe sur un sommet gris)
    //             Retourner true
    // Si color[v] == 0 → Appel récursif. Si true, retourner true.
    // TODO 3 : color[u] = 2 (noir – terminé)
```

```
} // TODO 4 : Retourner false (pas de cycle depuis u)
```

L'astuce des 3 couleurs : Blanc = pas encore vu. Gris = en train d'être exploré (dans la pile d'appels). Noir = complètement exploré. Si on tombe sur un gris, on a trouvé un cycle ! Un noir n'est pas un problème (c'est juste un chemin déjà exploré).

Test : prerequisites.txt n'a PAS de cycle (c'est un DAG). Mais prerequisites_cycle.txt ajoute l'arête Projet → Maths, et hasCycle() devrait retourner true.

Checkpoint étape 6 : make test — les tests 19 et 20 doivent passer. Le tri topologique respecte tous les prérequis ET le cycle est détecté dans prerequisites_cycle.txt. Bravo, c'est fini !

PIÈGES À ÉVITER

PIÈGE #1 : Oublier le double ajout pour les graphes non orientés

Symptôme : *BFS ne trouve pas tous les voisins*

```
// x FAUX :  
adjList[u].push_back(v); // manque l'autre sens !  
  
// ✓ CORRECT :  
adjList[u].push_back(v);  
if (!directed) adjList[v].push_back(u);
```

PIÈGE #2 : Utiliser une pile au lieu d'une file pour BFS

Symptôme : *L'ordre de visite est celui de DFS, pas de BFS*

```
// x FAUX :  
stack<int> s; // ← C'est du DFS, pas du BFS !  
  
// ✓ CORRECT :  
queue<int> q; // ← File FIFO = BFS
```

PIÈGE #3 : Oublier de marquer la source comme visitée AVANT la boucle

Symptôme : *La source est revisitée, distances fausses*

```
// x FAUX :  
q.push(source); // distance[source] pas initialisé !  
  
// ✓ CORRECT :  
distance[source] = 0;  
q.push(source); // ← source marquée PUIS ajoutée
```

PIÈGE #4 : DFS : confondre gris et noir pour la détection de cycles

Symptôme : *Faux positifs : des cycles détectés dans un DAG*

```
// x FAUX :  
if (visited[v]) return true; // FAUX : noir ≠ cycle  
  
// ✓ CORRECT :  
if (color[v] == 1) return true; // gris = cycle  
// color[v] == 2 (noir) → pas de problème
```

PIÈGE #5 : Tri topologique : trier par discovery au lieu de finish

Symptôme : *L'ordre ne respecte pas les prérequis*

```
// x FAUX :  
sort par discovery croissant // ← FAUX  
  
// ✓ CORRECT :  
sort par finish DÉCROISSANT // ← CORRECT
```

TESTS, BARÈME ET RENDU

COMPILATION ET TESTS

```

Compiler      : make
Exécuter     : ./bin/main
Tests        : make test && ./bin/test_tp1
Sortie       : 20 tests réussis sur 20
  
```

BARÈME DÉTAILLÉ

Étape	Fonction(s)	Tests	Points	Ce qu'on vérifie
1	Constructeur + addEdge + load + accesseurs	5	3	Graphe correctement construit
2	getAdjMatrix + printAdjList + printAdjMatrix	3	2	Deux représentations cohérentes
3	bfs + getPath	4	5	BFS correct, distances, chemin
4	connectedComponents + degreeSeparation	3	3	3 composantes + degrés de séparation
5	dfs + dfsVisit + dfsComplete	3	5	DFS correct, discovery/finish
6	topologicalSort + hasCycle	2	4	Tri valide, cycles détectés
—	Qualité du code	—	+3	Commentaires, nommage, clarté
TOT		20	25	

FORMAT DE RENDU

```

Fichier   : NOM_Prenom_TP1.zip
Contenu   : src/graph.cpp + src/bfs.cpp + src/dfs.cpp + capture_tests.png
NE PAS INCLURE : fichiers .o, bin/, obj/
  
```

CHECKLIST AVANT DE RENDRE

- ☐ Les 6 étapes sont implémentées
- ☐ make compile sans erreurs ni warnings (-Wall -Wextra)
- ☐ ./bin/test_tp1 affiche 20/20 tests réussis
- ☐ Le code est commenté (surtout : expliquer BFS et DFS)
- ☐ Nom/Prénom en haut de chaque fichier .cpp
- ☐ Le ZIP contient les 3 fichiers .cpp + capture

CONSEILS POUR RÉUSSIR

- 1. Étape 1 d'abord !** Le graphe est la fondation. Sans lui, rien ne marche.
- 2. Testez après chaque étape :** make test à chaque fonction complétée.
- 3. BFS = file, DFS = pile (ou récursion) :** ne les confondez JAMAIS.
- 4. Dessinez le graphe à la main :** tracez les 10 sommets et 15 arêtes, ça aide énormément.
- 5. Pour le debug :** affichez distance[] et parent[] à chaque étape de BFS.
- 6. Le tri topologique utilise finish[] :** pas discovery[]. Trier par finish DÉCROISSANT.
- 7. Les 3 couleurs pour les cycles :** Blanc → Gris → Noir. Cycle = retomber sur Gris.
- 8. N'hésitez pas à demander de l'aide !**

Bon courage ! BFS et DFS sont les deux parcours fondamentaux de tout graphe. Maîtrisez-les, et vous avez les bases pour Dijkstra, Kruskal, Bellman-Ford, et tout le reste.